

ComfyLAB: Guía de Inicio Rápido y Visión General Completa

ComfyLAB (**Com**fortable **Lab** **A**utomation **B**locks) es un entorno visual basado en bloques para la automatización de bancos de prueba e instrumentos de laboratorio científico, adquisición de datos y control experimental.

1. Introducción y Filosofía

1.1 ¿Qué es ComfyLAB?

ComfyLAB fue desarrollado para cerrar la brecha entre la programación tradicional mediante scripts (por ejemplo, scripts ad-hoc en Python) y los entornos visuales propietarios tradicionales (como LabVIEW). Proporciona un lienzo visual interactivo en el navegador web donde los instrumentos, las operaciones matemáticas, las rutinas de procesamiento de señales, las estructuras de control de flujo y los widgets de visualización en tiempo real se representan mediante **bloques interconectados**.

1.2 Arquitectura Dual: Interfaz Web UI + Motor de Ejecución Python

ComfyLAB opera mediante un modelo desacoplado cliente-servidor: 1. **Frontend (Interfaz de Usuario)**: Construido con React, TypeScript y Vite, ejecuta de forma fluida en cualquier navegador web moderno (local o remotamente). Ofrece manipulación de canvas de alto rendimiento, widgets de gráficos interactivos, diagramas en tiempo real y soporte multilingüe completo (portugués, inglés y español). 2. **Backend (Motor de Ejecución)**: Un servidor FastAPI de alto desempeño en Python 3.10+ gestiona la comunicación directa por hardware (mediante PyVISA y puertos serie), orquesta el grafo acíclico dirigido (DAG) de ejecución, administra hilos asíncronos en tiempo real y aísla la ejecución de scripts.

1.3 Seguridad y Protección Integradas

- Rutinas de Desconexión de Emergencia de Hardware**: Cuando un flujo de trabajo se detiene deliberadamente o sufre un error de ejecución crítico, ComfyLAB activa automáticamente rutinas de parada segura registradas en los controladores de hardware (por ejemplo, apagando láseres de bombeo, deshabilitando salidas de alta tensión y apagando amplificadores) para salvaguardar la integridad física de los equipos del laboratorio.
- Aislamiento de Scripts y Protección contra Blueprints No Confiables**: Los flujos importados que contienen código ejecutable personalizado solicitan confirmación explícita al usuario antes de permitir su ejecución.
- Autenticación Ligera para Acceso Remoto**: Permite la supervisión remota de ensayos a través de tokens seguros de dos palabras, evitando la exposición no autorizada de la red local del laboratorio.

2. Capacidades y Flexibilidad: Qué Puede (y Qué No Puede) Hacerse

2.1 Qué Puede Hacerse con ComfyLAB

- **Automatización de Instrumentos VISA y SCPI:** Conexión plug-and-play con osciloscopios, fuentes de alimentación, multímetros digitales, generadores de señales arbitrarias, analizadores de espectro óptico (OSA), controladores de temperatura y etapas de traslación mediante USB, GPIB, Ethernet o RS-232.
- **Barridos de Parámetros Complejos:** Coordinación de bucles anidados de barrido de longitud de onda, tensión, corriente o frecuencia mientras se adquieren trazas de múltiples canales de forma síncrona.
- **Procesamiento de Señal y Matemáticas en Vivo:** Transformadas rápidas de Fourier (FFT), filtrado digital, ajuste de curvas no lineales, estadísticas en tiempo real y operaciones matriciales sin latencia perceptible.
- **Inyección de Código Multilenguaje:** Inserción de bloques de código en vivo en 9 lenguajes (Python, Rust, Julia, R, JavaScript, TypeScript, Lua, GNU Octave y Wolfram) dentro del mismo flujo de datos.
- **Instrumentos Virtuales Embebidos para Pruebas Offline:** Ejecución de simulaciones y desarrollo de blueprints fuera del laboratorio mediante generadores virtuales y osciloscopios simulados incorporados.
- **Paneles de Control para Operadores:** Conmutación instantánea al **Dashboard del Operador** para ocultar el cableado técnico y presentar únicamente pantallas numéricas, perillas, botones y gráficos para uso diario.

2.2 Limitaciones Realistas: Qué No Puede (o No Debe) Hacerse

- **Control Determinista Duro en Tiempo Real (Hard Real-Time < 1 ms):** ComfyLAB opera sobre sistemas operativos estándar de escritorio (Linux, Windows, macOS). No sustituye sistemas basados en FPGA o microcontroladores dedicados para bucles de control de microsegundos cerrados por hardware.
- **Adquisición a Frecuencias Gigahertz en Bruto en Streaming Continuo:** Mientras que los instrumentos transfieren formas de onda procesadas a través de VISA a altas tasas, el streaming continuo de gigabytes por segundo directamente a través de sockets web no es el objetivo de diseño del motor visual.

3. Comparación con Paradigmas Alternativos

Característica	Python Puro (Scripts)	LabVIEW™	MATLAB™ / Simulink	ComfyLAB
Costo de Licencia	Gratuito / Código Abierto	Muy Costoso (Anual/Por PC)	Muy Costoso (Por Módulo)	100% Gratuito y Libre (GPLv3)
Curva de Entrada	Requiere programar código	Programación gráfica propietaria	Requiere sintaxis de matrices	Flujo visual intuitivo por bloques
Control de Versiones	Excelente (texto puro)	Difícil (binarios opacos)	Mixto	Excelente (Blueprints JSON)
Código Personalizado	Nativo	Nodos C/Python complejos	S-Functions complejas	Nodos de script en 9 lenguajes
Despliegue	Requiere entorno Python	Instalador pesado	Runtime voluminoso	PyPI, Binario único o Web
Bloqueos de Hardware	Implementación manual	DAQmx propietario	Instrument Toolbox	Gestor asíncrono VISA nativo

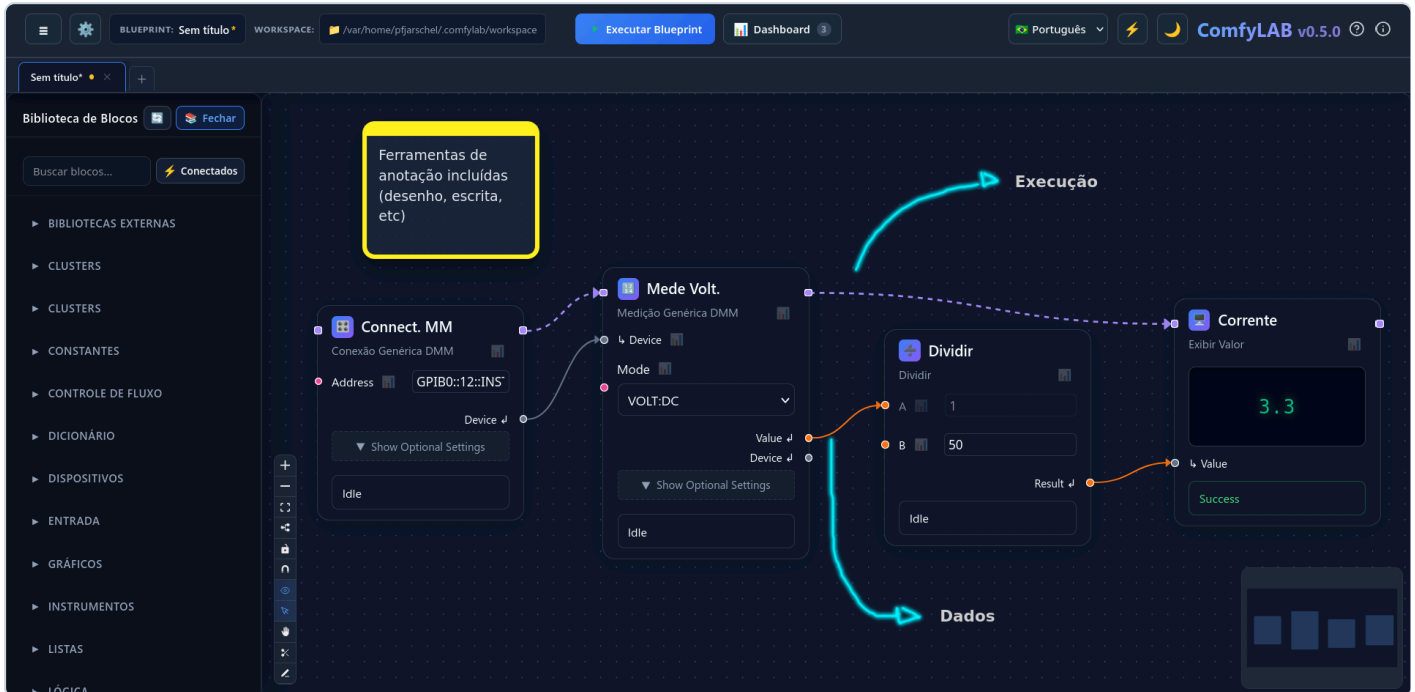
Conclusiones Principales:

- Frente a Python Puro:** Programar scripts en Python es potente, pero diseñar interfaces gráficas, lazos seguros de control de hardware y gráficos interactivos exige cientos de líneas de código repetitivo. ComfyLAB mantiene la potencia de Python ofreciendo una interfaz visual inmediata.
- Frente a LabVIEW™:** LabVIEW sufre de costos de licencia elevados por puesto, archivos binarios que impiden un control de versiones limpio y dependencia de un único proveedor. ComfyLAB ofrece una alternativa moderna, abierta y basada en estándares web abiertos.
- Frente a MATLAB™:** MATLAB destaca en análisis matricial, pero impone severas restricciones presupuestarias en laboratorios multiusuario. ComfyLAB utiliza backends científicos estándar libres (NumPy, SciPy).

4. Diseño de la Interfaz y Conceptos Fundamentales

4.1 Visión General del Diseño de la Interfaz

La interfaz de ComfyLAB está compuesta por tres áreas funcionales primarias rodeadas por controles superiores:



Pantalla general de ComfyLAB

Descripción Detallada de la Interfaz:

- **Barra de Herramientas Superior:** Controles para ejecutar (▶ / Ctrl+R), pausar (⏸) y detener (⏹ / Ctrl+Shift+R) blueprints, abrir el **Dashboard del Operador** (📊 / D), guardar y cargar workspaces (Ctrl+S), alternar temas visuales y configurar ajustes globales de VISA y diagnósticos.
- **Barra Lateral (Izquierda):** Catálogo organizado y con barra de búsqueda que contiene todos los bloques nativos, bloques de instrumentos virtuales, bloques de scripts y agrupaciones personalizadas (clusters).
- **Lienzo Central (Canvas):** Área de trabajo visual infinita donde se colocan, organizan y conectan los bloques funcionales mediante cables virtuales.
- **Inspector de Bloques (Derecha):** Panel contextual que se despliega al seleccionar un bloque. Muestra parámetros de configuración, valores de pines en tiempo real, consola de salida y documentación rápida.
- **Pestañas y Rutas de Navegación:** Permiten alternar entre múltiples archivos de blueprint abiertos y explorar el interior de macro-bloques o clusters.

4.2 Pines y Conexiones: Flujo de Ejecución vs. Valores de Datos

Las conexiones entre bloques en ComfyLAB se dividen estrictamente en dos categorías:

PIN DE EJECUCIÓN (Blanco / Forma de Flecha ►)
Determina CUÁNDO se ejecuta un bloque. Orden secuencial estricto
PINES DE DATOS (Círculos Coloreados ●)
Transportan VALORES (números, matrices, cadenas, configuraciones)

Referencia de Colores de los Pines de Datos:

- **Azul Claro (●):** Números escalares (`float` , `int`).
- **Verde (●):** Matrices numéricas y vectores (`numpy.ndarray`).
- **Amarillo (●):** Cadenas de texto (`string`).
- **Púrpura (●):** Booleano (`true` / `false`).
- **Naranja (●):** Objetos estructurados, diccionarios y recursos VISA de instrumentos.

4.3 Lógica de Ejecución y Cadenas Independientes

ComfyLAB detecta automáticamente cadenas de ejecución secuenciales y ramificaciones independientes. Dos o más experimentos o lazos de instrumentación pueden ejecutarse en paralelo en el mismo lienzo sin interferencias de estado.

4.4 Recursos Avanzados

- **Macro-bloques (Clusters):** Posibilidad de agrupar sub-circuitos enteros en un único bloque visual limpio para mantener ordenados los diagramas de gran escala.
- **Lienzo de Anotaciones (Whiteboard Overlay):** Permite dibujar diagramas, notas a mano alzada y esquemas ópticos directamente sobre el lienzo de trabajo.

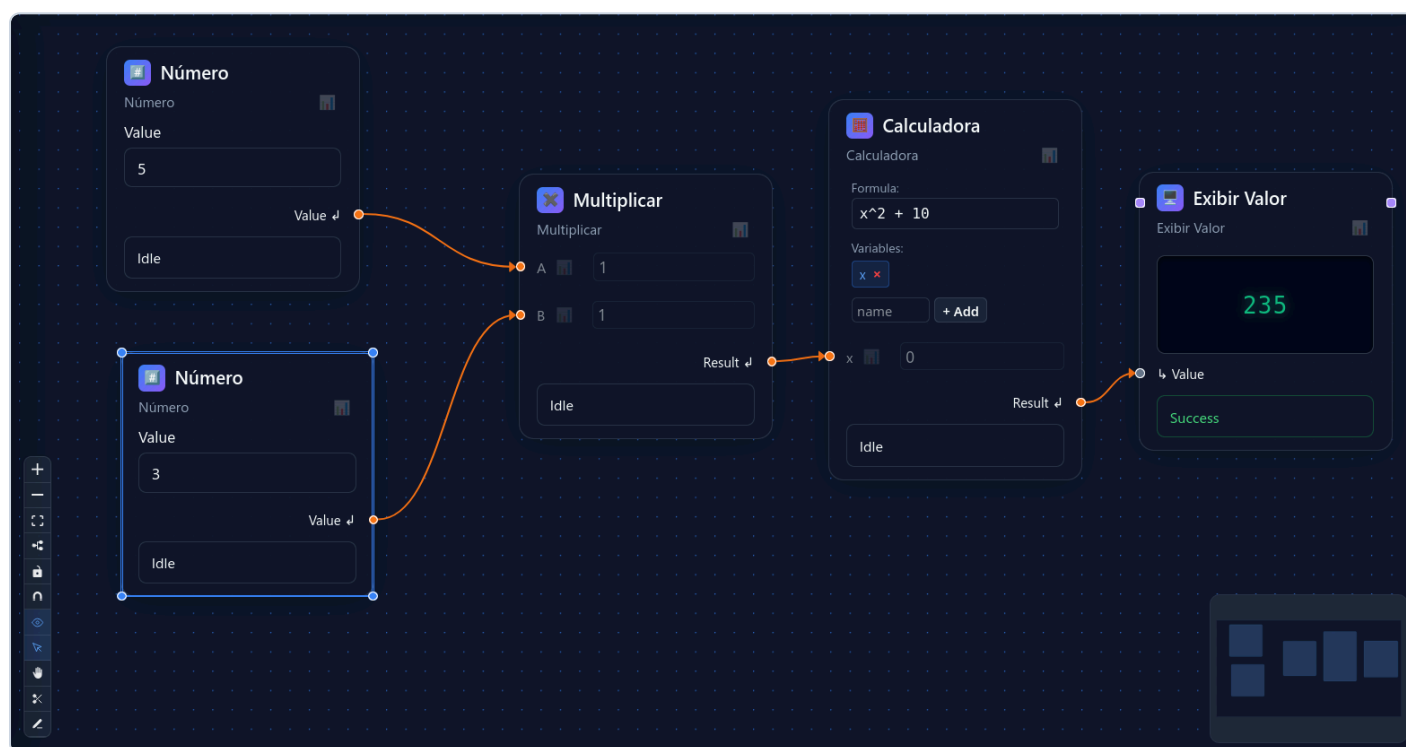
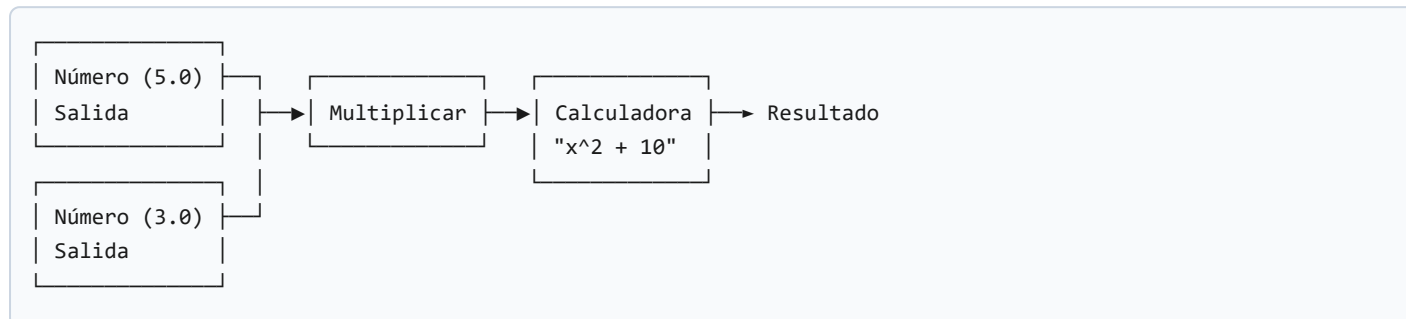
5. Resumen de las Categorías de Bloques Nativos

1. **Constantes y Entradas:** Bloques para números escalares, valores booleanos, cadenas de texto, deslizadores interactivos y matrices constantes.
2. **Matemáticas y Operaciones:** Operaciones aritméticas básicas, trigonometría, álgebra matricial, redondeo y bloque Calculadora con evaluador de fórmulas.
3. **Flujo de Control:** Bloques de inicio de ejecución, bucles `For`, lazos condicionales `While`, bifurcaciones `If/Else` y retardos temporales sincronizados.
4. **Matrices Numéricas:** Generación de vectores `linspace`, `arange`, concatenación, filtrado, extracción de sub-matrices y estadísticas.
5. **Procesamiento de Señales:** Transformadas rápidas de Fourier (FFT), filtros pasa-bajas/altas, interpolación y detección de picos.
6. **Gráficos y Visualización:** Gráficos XY en tiempo real, gráficos multicanal, histogramas, diagramas polares, vistas de tablas y tarjetas de operador.
7. **Instrumentos VISA y Hardware:** Conexiones de puerto, comandos de escritura/lectura SCPI, generadores de funciones, fuentes y osciloscopios.
8. **Instrumentos Virtuales:** Dispositivos de prueba emulados para ensayar rutinas sin requerir hardware físico conectado.
9. **Entrada/Salida de Archivos:** Lectura y exportación automática a formatos estándar CSV, JSON, HDF5 e imágenes.
10. **Bloques de Script:** Módulos editables en vivo en Python, Rust, Julia, R, JavaScript, Lua y Octave.

6. Tutoriales Prácticos de Iniciación

Tutorial 1: Pipeline Matemático e Inspección en Vivo

Objetivo: Crear dos valores escalares, multiplicarlos, evaluar una fórmula matemática con la Calculadora e inspeccionar el resultado en tiempo real.



Ejemplo 1

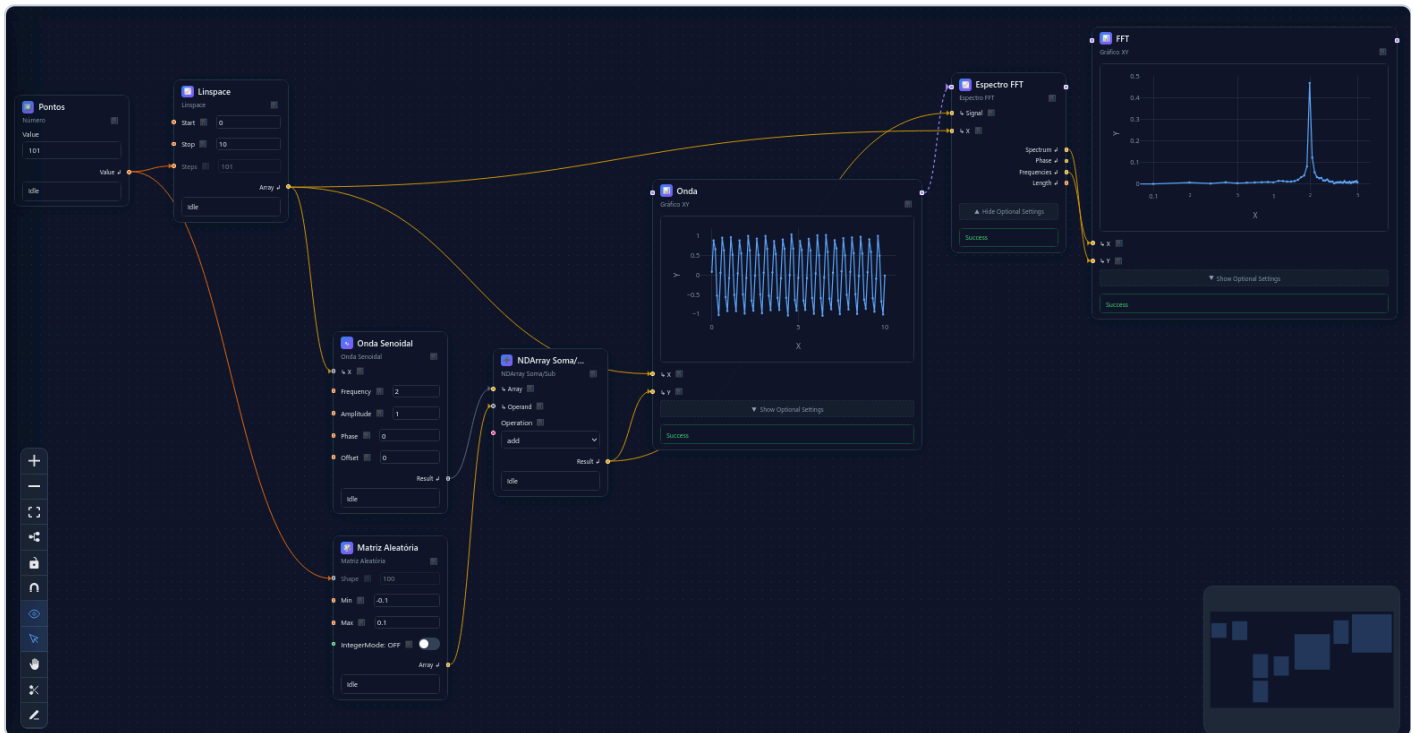
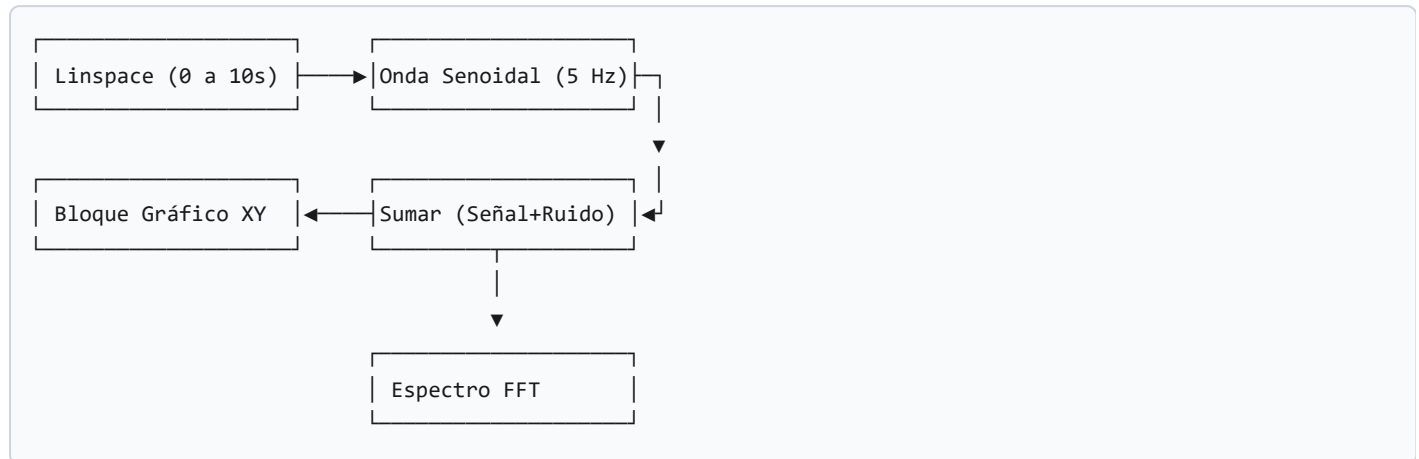
Instrucciones Paso a Paso:

1. Abra ComfyLAB y cree un nuevo workspace/blueprint.
2. En la **Barra Lateral** izquierda, busque el bloque **Número** (`Number` , en *Constantes*) y arrastre **dos** bloques al lienzo.
3. Establezca el valor del primer bloque en `5.0` y el del segundo en `3.0` .
4. Arrastre un bloque **Multiplicar** (`Multiply` , en *Matemáticas / Básica*) al lienzo.
5. Conecte la salida de datos del Número 1 a la Entrada A de Multiplicar, y la del Número 2 a la Entrada B.
6. Arrastre un bloque **Calculadora** (`Calculator` , en *Matemáticas / Básica*). En el Inspector, configure la expresión como `x^2 + 10` y agregue la variable `x` .
7. Conecte la salida de Multiplicar (`15.0`) a la entrada `x` de la Calculadora.
8. Haga clic en **Ejecutar Blueprint** ( o `Ctrl+R`) en la barra superior.
9. Haga clic en el bloque Calculadora para abrir el **Inspector de Bloques** a la derecha.

10. **Verificación:** En el Inspector, confirme el valor en el pin de salida **Resultado** (`result`). Debe indicar $(15)^2 + 10 = 235.0$.

Tutorial 2: Generación de Señal Sintética y Gráfico en Vivo

Objetivo: Generar un vector de onda senoidal ruidosa, calcular su espectro FFT y graficarlo en tiempo real.



Ejemplo 2

Instrucciones Paso a Paso:

1. Busque **Linspace** (en *Matrices Numéricas*) en la Barra Lateral. Configure **Start = 0**, **Stop = 10**, **Points = 1000** para crear el vector de tiempo.
2. Busque **Onda Senoidal** (Sine Wave, en *Matemáticas / Funciones*). Conecte la salida de Linspace a la entrada **X** de la Onda Senoidal. Ajuste **Frecuencia = 5.0 Hz**.
3. Agregue un bloque **Matriz Aleatoria** (Random Array, en *Matemáticas / Aleatorio*, ajuste **Forma = 1000**, **Mín = -0.2**, **Máx = 0.2**) y un bloque **Sumar** (Add) para incorporar ruido.
4. Arrastre un bloque **Gráfico XY** (XY Plot, en *Gráficos*) al lienzo.
5. Conecte el vector de tiempo de Linspace al pin **X** del Gráfico XY y la señal ruidosa al pin **Y**. Conecte un cable de ejecución al pin **Graficar** (Plot).
6. (Opcional) Conecte la señal a un bloque **Espectro FFT** (FFT Spectrum, en *Procesamiento de Señales*) para analizar los componentes espectrales.

7. Haga clic en **Ejecutar Blueprint** ( o Ctrl+R).

8. **Verificación:** El widget del Gráfico XY mostrará de inmediato la forma de onda. ¡Use la rueda del mouse para hacer zoom y arrastre para desplazarse por los ejes!

Tutorial 3: Barrido Automatizado de Parámetros y Registro en CSV

Objetivo: Ejecutar un barrido de frecuencia por pasos mediante un bucle, emular una medición física y guardar automáticamente los datos en un archivo CSV.

Instrucciones Paso a Paso:

1. Arrastre un bloque **Bucle For** (`For Loop` , en *Flujo de Control*) al lienzo. Defina `Inicio = 100` , `Fin = 1000` , `Paso = 50` .
2. Conecte el pin de ejecución del lazo a un bloque de medición o generador de funciones.
3. Conecte la salida del índice actual del barrido y el dato medido a un bloque **Construir Fila** (`Append Row`).
4. Conecte la salida de la matriz acumulada a un bloque **Escribir CSV** (`Write CSV` , en *Entrada/Salida de Archivos*). Configure la ruta del archivo deseado.
5. Inicie la ejecución. Observe cómo el bucle avanza paso a paso de forma coordinada y los datos se registran en el disco de manera persistente.

Tutorial 4: Creación y Publicación de un Bloque de Script Personalizado

Objetivo: Escribir una función en Python personalizada dentro de ComfyLAB y empaquetarla como un bloque reutilizable en la paleta.

Instrucciones Paso a Paso:

1. En la Barra Lateral, busque **Script Python** (`Python Script`) y arrástrelo al lienzo.
2. Seleccione el bloque y abra el editor de código integrado en el Inspector o mediante doble clic.
3. Escriba su función, por ejemplo:

```
def run(inputs):  
    val = inputs.get("val", 0.0)  
    return {"squared": val ** 2}
```

4. Agregue los pines correspondientes en la interfaz del bloque: entrada `val` (tipo número) y salida `squared` (tipo número).
5. Pruebe su bloque conectando una entrada y ejecutando el flujo.
6. Haga clic en **Publicar como Bloque Personalizado** para que quede disponible permanentemente en la paleta para futuros blueprints.

Tutorial 5: Conexión con Hardware VISA (Consulta SCPI)

Objetivo: Conectar con un instrumento físico por USB/GPIB, enviar el comando de identificación `*IDN?` y visualizar el modelo y firmware del equipo.

Instrucciones Paso a Paso:

1. Conecte el instrumento al ordenador del laboratorio mediante USB, Ethernet o conversor GPIB.
2. Arrastre un bloque **Abrir Recurso VISA** (`VISA Open Resource` , en *VISA / Hardware*).
3. Seleccione el recurso detectado en la lista desplegable (por ejemplo, `USB0::0x1AB1::0x04CE::DS1ZA...::INSTR`).
4. Conecte la referencia del instrumento a un bloque **Consulta VISA** (`VISA Query`). Configure el comando como `*IDN?` `\n` .
5. Conecte la salida de texto a una tarjeta de texto o visualizador en pantalla.
6. Ejecute el blueprint y verifique la cadena devuelta por el instrumento en la pantalla.

Tutorial 6: Automatización Offline con Instrumentos Virtuales Embebidos

Objetivo: Practicar y diseñar rutinas completas de instrumentación en una laptop o PC sin necesidad de equipos físicos conectados.

Instrucciones Paso a Paso:

1. En la Barra Lateral, explore la sección **Instrumentos Virtuales** (**Virtual Instruments**).
2. Agregue un **Generador de Funciones Virtual** y un **Osciloscopio Virtual de 2 Canales**.
3. Conecte las salidas emuladas del generador a las entradas del osciloscopio virtual.
4. Active el blueprint: el osciloscopio virtual comenzará a sincronizar formas de onda sintetizadas exactamente como si provinieran de hardware real.
5. Esto permite a estudiantes e investigadores preparar sus protocolos de medición en casa antes de acudir al banco de pruebas físico del laboratorio.

7. Guía Rápida de Atajos y Referencia de Teclado

Atajos de Teclado

- **Ctrl + R** : Ejecutar Blueprint activo.
- **Ctrl + Shift + R** : Detener ejecución de inmediato (parada segura de hardware).
- **Ctrl + S** : Guardar Workspace actual en archivo.
- **Ctrl + Z / Ctrl + Y** : Deshacer / Rehacer cambios en el lienzo.
- **D** : Alternar vista del **Dashboard del Operador**.
- **Espacio + Arrastre** : Desplazamiento panorámico (Pan) por el lienzo.
- **Rueda del Mouse** : Zoom interactivo sobre el lienzo o sobre los widgets de gráficos.
- **Supr / Backspace** : Eliminar bloque o conexión seleccionada.
- **Ctrl + G** : Agrupar bloques seleccionados en un Cluster.